CONSTRUCTOR

1.A constructor is a special type of method that is called instantiates an object using definition found in the class. it is normally used to initialize instance variables.
2.A class function that begins with double underscore(___). the name of the constructor is always same __init__().
3.while creating object to a class can accept arguments if necessary.
4.constructor should not return any value.
5.constructor is called when class object is created.
6.constructor always represented in public.

there are two types:

(a)default constructor:

A constructor without parameters is called 'default constructor' 'zero parameterized constructor'. Every object is initialized with same data.

Syntax:

```
class class_name:
    def __init__(self):
        statements
```

Example:

```
class person:
  def __init__(self):#default constructor
    self.name="sushma"
    self.desg="trainer"
  def display(self):
    print("person name:",self.name)
    print("person designation:",self.desg)
p=person()#object1
p.display()

p1=person()#object2
p1.display()
```

2.parameterized constructor:

A constructor with parameters is called "parameterized constructor'. here, every object is initialized with different data.

Syntax:

```
class class_name:
```

```
    def __init__(self,arg1,arg2,.....):
        statements
```

Example:

```
class person:
  def __init__(self,name,desg):#parameterized constructor
    self.name=name
    self.desg=desg
  def display(self):
    print("person name:",self.name)
    print("person designation:",self.desg)
p=person("pooja","hr")#object1
p.display()

p1=person("harsha","admin")#object2
p1.display()
```

Access modifiers:

it play an important role to protect the data from unauthorized access as well as protecting it from getting manipulated.
there are 3 types:

1.public:

- **public members are accessible from outside the class through an object of the class.**
- **by default, all the variables are member functions of a class are 'public' in a python.**

Example:

```
class Employee:
  def __init__(self,name,sal):
    self.name=name#public variables
    self.sal=sal
e=Employee("sushma",40000)
print(e.sal)
print(e.name)
```

2.private:

- **these members are only accessible from within the class not outside of the class.**

- **while the addition of prefix__(double underscore) result in a member variable or function becoming private.**

Example:

```
class Employee:
   def __init__(self,name,sal):
      self.__name=name#private variables
      self.__sal=sal
e=Employee("sushma",40000)
print(e.__sal)
print(e.__name)
```

3.protected:

- **protected members are accessible from outside of the class but only in a class derived from it is called child class.**
- **Adding a prefix _(single underscore) to variable name makes it protected.**

Example:

```
class Employee:#super class
   def __init__(self,name,age):
      self._name=name
      self._age=age
class HR(Employee):#sub class
   def task(self):
      print("assiging work to employee")
e=Employee("pooja",23)
print(e._name)
print(e._age)
h=HR("mani",32)
print(h._name)
print(h._age)
h.task()
```